

Parcial final electiva Django

Basado en el orden la lista se organiza la sala, en ella los pares de la lista realizan el ejercicio 2 y los impares realizan el ejercicio 1, realice cada uno de los requerimientos descritos a continuación.

EJERCICIO 1: Sistema de Gestión de Reservas de Laboratorios

Objetivo general

Desarrollar una aplicación Django que gestione las reservas de laboratorios, integrando autenticación, control de roles (docente y administrador) y validación de disponibilidad horaria (*nombre DEL PROYECTO_NOMBRE_ESTUDIANTE, nombre de las aplicaciones_NOMBRE_ESTUDIANTE*).

Requerimientos funcionales

1. Autenticación y roles

- Login y logout usando el sistema de usuarios de Django.
- Roles:
 - **Docente:** crea, edita y cancela sus propias reservas.
 - **Administrador:** aprueba o rechaza cualquier reserva.

2. Gestión de reservas (CRUD)

- Validar que no existan conflictos de horarios en el mismo laboratorio.
- Permitir edición o eliminación solo si la reserva está en estado “Pendiente”.
- El administrador puede cambiar el estado a “Aprobada” o “Rechazada”.

3. Visualización y filtros

- Listar reservas filtradas por fecha o laboratorio.
- Mostrar estadísticas de uso y exportación a CSV.

Requerimientos técnicos

- Uso de vistas basadas en clases (ListView, CreateView, UpdateView, DeleteView).
- Control de acceso con LoginRequiredMixin y UserPassesTestMixin.
- Diseño modular con plantillas base y estilos.

Modelo: Reserva

Campo	Tipo de dato	Descripción
usuario	ForeignKey(User, on_delete=models.CASCADE)	Usuario que realiza la reserva.
laboratorio	CharField(max_length=100)	Nombre del laboratorio solicitado.
fecha	DateTimeField()	Fecha de la reserva.
hora_inicio	TimeField()	Hora de inicio de la reserva.
hora_fin	TimeField()	Hora de finalización de la reserva.
estado	CharField(max_length=20, choices=[('pendiente', 'Pendiente'), ('aprobada', 'Aprobada'), ('rechazada', 'Rechazada')])	Estado actual de la reserva.
motivo	TextField()	Motivo o descripción de la reserva.
fecha_creacion	DateTimeField(auto_now_add=True)	Fecha en que se crea la reserva.

EJERCICIO 2: Sistema de Seguimiento de Proyectos Académicos

Objetivo general

Implementar una aplicación Django que permita registrar proyectos académicos, gestionarlos mediante distintos estados, y permitir la comunicación entre docentes y estudiantes con comentarios y notificaciones por correo electrónico (*nombre DEL PROYECTO_NOMBRE_ESTUDIANTE, nombre de las aplicaciones_NOMBRE_ESTUDIANTE*).

Requerimientos funcionales

1. Autenticación y roles

- Autenticación mediante login/logout.
- Roles:
 - **Estudiante:** crea y gestiona sus proyectos.

- **Docente:** revisa, comenta y aprueba proyectos.

2. Gestión de proyectos (CRUD)

- Los estudiantes pueden crear, actualizar y eliminar proyectos propios.
- Los docentes pueden cambiar el estado y asignar calificación.
- Cuando el estado cambia a “Aprobado”, se bloquea la adición de nuevos comentarios.

3. Gestión de comentarios

- Cada comentario debe registrar el usuario, fecha y texto.
- Al crear un comentario, se envía una notificación por correo electrónico al estudiante.

4. Visualización

- Filtrar proyectos por estado o estudiante.
- Exportar la lista en formato CSV o PDF.

Requerimientos técnicos

- Uso de django-crispy-forms y vistas basadas en clases.
- Decoradores de permiso y control de grupos.
- Configuración de envío de correo (EmailMessage o send_mail).

Modelo principal: Proyecto

Campo	Tipo de dato	Descripción
titulo	CharField(max_length=200)	Título del proyecto.
descripcion	TextField()	Descripción o resumen del proyecto.
estudiante	ForeignKey(User, related_name='proyectos', on_delete=models.CASCADE)	Usuario que registra el proyecto.
documento	FileField(upload_to='proyectos/')	Archivo asociado (por ejemplo, PDF).
estado	CharField(max_length=20, choices=[('enviado', 'Enviado'), ('revisión', 'En Revisión'), ('aprobado', 'Aprobado')])	Estado actual del proyecto.

Campo	Tipo de dato	Descripción
fecha_envio	DateTimeField(auto_now_add=True)	Fecha de creación del registro.
fecha_revision	DateTimeField(null=True, blank=True)	Fecha de la última revisión (opcional).
calificacion	DecimalField(max_digits=4, decimal_places=2, null=True, blank=True)	Calificación asignada (opcional).

CRITERIOS DE EVALUACION

Criterio de evaluación	Descripción	Puntaje (0.0 – 5.0)
1. Autenticación y control de roles	Implementación del sistema de login/logout y restricción de acceso por tipo de usuario (docente/administrador).	1.5
2. CRUD y validaciones del modelo Reserva	Correcta creación, edición, eliminación y validación de disponibilidad de horarios en el laboratorio.	1.5
3. Filtros, reportes y visualización	Implementación de filtros por fecha o laboratorio, exportación a CSV y presentación organizada de los datos.	1.0
4. Calidad técnica y presentación	Organización del código, estructura de plantillas, estilo visual y documentación básica.	1.0
TOTAL		5.0